PROGRAM

```
import statistics

def central_tendency_and_dispersion(data):
    if not data:
        return "Data list is empty"

    # Calculating mean
    mean = statistics.mean(data)

    # Calculating median
    median = statistics.median(data)

    # Calculating mode
    try:
        mode = statistics.mode(data)
    except statistics.StatisticsError:
        mode = "No unique mode found"

    # Calculating variance
    variance = statistics.variance(data)

    # Calculating standard deviation
    std_dev = statistics.stdev(data)

    # Displaying the results
    print(f"Mean: {mean}")
    print(f"Median: {median}")
    print(f"Mode: {mode}")
```

```
    print(f"Variance: {variance}")
    print(f"Standard Deviation: {std_dev}")


# Example usage:
data = [1, 2, 2, 3, 4, 4, 4, 5, 5]
central_tendency_and_dispersion(data)
```

OUTPUT:

Mean: 3.3333333333333335

Median: 4

Mode: 4

Variance: 2

Standard Deviation: 1.4142135623730951

PROGRAM:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.datasets import fetch_california_housing

# Load the California Housing dataset
california = fetch_california_housing()

# Convert the dataset into a DataFrame
california_df = pd.DataFrame(california.data, columns=california.feature_names)
california_df['MedHouseVal'] = california.target

# Display the first few rows of the DataFrame
print("First few rows of the dataset:")
print(california_df.head())

# Implement Linear Regression using 'AveRooms' to predict 'MedHouseVal'
X = california_df[['AveRooms']]
y = california_df['MedHouseVal']

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Create and train the linear regression model
```

```
linear_regressor = LinearRegression()
linear_regressor.fit(X_train, y_train)

# Make predictions on the testing set
y_pred = linear_regressor.predict(X_test)

# Evaluate the model
mse_linear = mean_squared_error(y_test, y_pred)
r2_linear = r2_score(y_test, y_pred)
print(f'\nLinear Regression MSE: {mse_linear}')
print(f'Linear Regression R-squared: {r2_linear}')

# Plot the regression line
plt.scatter(X_test, y_test, color='blue')
plt.plot(X_test, y_pred, color='red')
plt.xlabel('Average number of rooms (AveRooms)')
plt.ylabel('Median House Value (MedHouseVal)')
plt.title('Linear Regression: AveRooms vs MedHouseVal')
plt.show()

# Implement Multiple Linear Regression using all features to predict 'MedHouseVal'
X = california_df.drop('MedHouseVal', axis=1)
y = california_df['MedHouseVal']

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Create and train the multiple linear regression model
```

```
multiple_linear_regressor = LinearRegression()
multiple_linear_regressor.fit(X_train, y_train)

# Make predictions on the testing set
y_pred = multiple_linear_regressor.predict(X_test)

# Evaluate the model
mse_multiple = mean_squared_error(y_test, y_pred)
r2_multiple = r2_score(y_test, y_pred)
print(f'\nMultiple Linear Regression MSE: {mse_multiple}')
print(f'Multiple Linear Regression R-squared: {r2_multiple}')

# Plot the actual vs predicted prices
plt.scatter(y_test, y_pred, color='blue')
plt.xlabel('Actual Median House Value')
plt.ylabel('Predicted Median House Value')
plt.title('Multiple Linear Regression: Actual vs Predicted Prices')
plt.show()
```

OUTPUT:

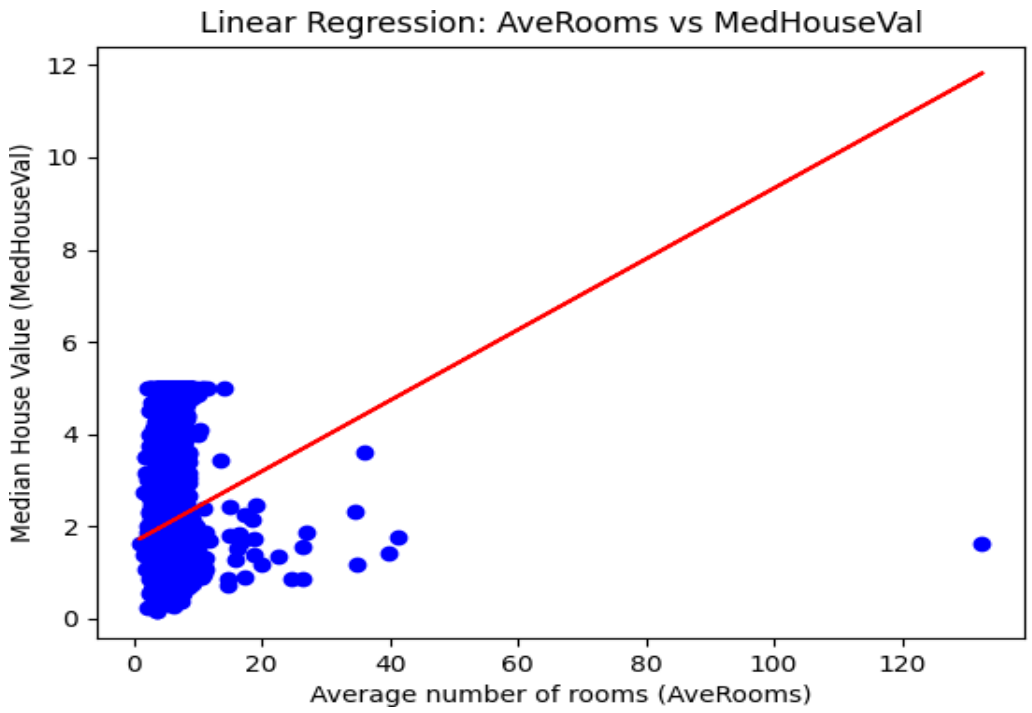
```
==== RESTART: C:/Users/ELCOT/AppData/Local/Programs/Python/Python310/aml2.py
First few rows of the dataset:
   MedInc  HouseAge  AveRooms  ...  Latitude  Longitude  MedHouseVal
0  8.3252      41.0  6.984127  ...     37.88    -122.23        4.526
1  8.3014      21.0  6.238137  ...     37.86    -122.22        3.585
2  7.2574      52.0  8.288136  ...     37.85    -122.24        3.521
3  5.6431      52.0  5.817352  ...     37.85    -122.25        3.413
4  3.8462      52.0  6.281853  ...     37.85    -122.25        3.422

[5 rows x 9 columns]

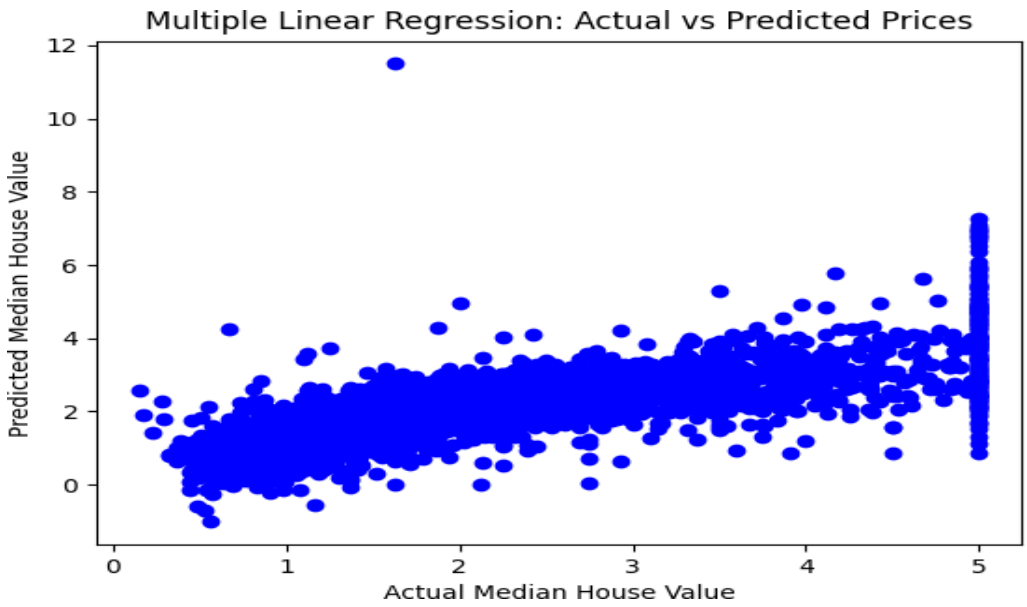
Linear Regression MSE: 1.2923314440807299
Linear Regression R-squared: 0.013795337532284901

Multiple Linear Regression MSE: 0.5558915986952437
Multiple Linear Regression R-squared: 0.5757877060324512
```

Linear regression:



Multiple Linear regression:



PROGRAM:

```
# Step 1: Import Necessary Libraries
import numpy as np
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# Step 2: Load the Data
iris = load_iris()
X = iris.data
y = iris.target

# Step 3: Preprocess the Data (split into training and test sets)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Step 4: Train the Model
log_reg = LogisticRegression(max_iter=200)
log_reg.fit(X_train, y_train)

# Step 5: Make Predictions
y_pred = log_reg.predict(X_test)

# Step 6: Evaluate the Model
accuracy = accuracy_score(y_test, y_pred)
conf_matrix = confusion_matrix(y_test, y_pred)
class_report = classification_report(y_test, y_pred)
```

```
print(f"Accuracy: {accuracy:.2f}")
print("Confusion Matrix:\n", conf_matrix)
print("Classification Report:\n", class_report)
```

OUTPUT:

```
Accuracy: 1.00
Confusion Matrix:
 [[19  0  0]
 [ 0 13  0]
 [ 0  0 13]]
Classification Report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        19
           1       1.00      1.00      1.00        13
           2       1.00      1.00      1.00        13

    accuracy                           1.00        45
   macro avg       1.00      1.00      1.00        45
weighted avg       1.00      1.00      1.00        45
```

PROGRAM:

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
from sklearn.datasets import load_breast_cancer

# Load dataset
data = load_breast_cancer()
X = pd.DataFrame(data.data, columns=data.feature_names)
y = pd.Series(data.target)

# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Feature scaling
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Train the Logistic Regression model
model = LogisticRegression()
model.fit(X_train, y_train)

# Make predictions on the test set
y_pred = model.predict(X_test)
```

```
# Evaluate the model
accuracy = accuracy_score(y_test, y_pred)
cm = confusion_matrix(y_test, y_pred)
report = classification_report(y_test, y_pred)

print(f'Accuracy: {accuracy}')
print('Confusion Matrix:')
print(cm)
print('Classification Report:')
print(report)
```

OUTPUT:

```
Accuracy: 0.9824561403508771
Confusion Matrix:
[[ 62   1]
 [  2 106]]
Classification Report:
              precision    recall  f1-score   support

           0       0.97      0.98      0.98        63
           1       0.99      0.98      0.99       108

    accuracy                           0.98       171
   macro avg       0.98      0.98      0.98       171
weighted avg       0.98      0.98      0.98       171
```

PROGRAM:

```
# Import necessary libraries
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score

# Load the Iris dataset
iris = load_iris()
X = iris.data
y = iris.target

# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# k-Nearest Neighbors classification
knn = KNeighborsClassifier(n_neighbors=3)
knn.fit(X_train, y_train)
y_pred_knn = knn.predict(X_test)

# Naive Bayes classification
nb = GaussianNB()
nb.fit(X_train, y_train)
y_pred_nb = nb.predict(X_test)

# Evaluate the models
print("k-Nearest Neighbors Accuracy:", accuracy_score(y_test, y_pred_knn
print("Naive Bayes Accuracy:", accuracy_score(y_test, y_pred_nb))
```

OUTPUT:

k-Nearest Neighbors Accuracy: 1.0

Naive Bayes Accuracy: 0.9777777777777777

PROGRAM:

```
import numpy as np
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# Load dataset
iris = load_iris()
X = iris.data
y = iris.target

# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Create a Decision Tree Classifier
dt = DecisionTreeClassifier()

# Set up the parameter grid for tuning
param_grid = {
    'criterion': ['gini', 'entropy'],
    'splitter': ['best', 'random'],
    'max_depth': [None, 10, 20, 30],
    'min_samples_split': [2, 10, 20],
    'min_samples_leaf': [1, 5, 10]
}
```

```
# Set up the GridSearchCV to tune the parameters
grid_search = GridSearchCV(estimator=dt, param_grid=param_grid, cv=5, scoring='accuracy')

# Fit the model
grid_search.fit(X_train, y_train)

# Print the best parameters found by GridSearchCV
print("Best Parameters: ", grid_search.best_params_)

# Predict the results using the best model
best_dt = grid_search.best_estimator_
y_pred = best_dt.predict(X_test)

# Evaluate the model
print("Accuracy: ", accuracy_score(y_test, y_pred))
print("Classification Report:\n", classification_report(y_test, y_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
```

OUTPUT:

```
Best Parameters:  {'criterion': 'gini', 'max_depth': 30, 'min_samples_leaf': 1, 'min_samples_split': 20, 'splitter': 'random'}
Accuracy:  0.9333333333333333
Classification Report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        19
           1       0.81      1.00      0.90        13
           2       1.00      0.77      0.87        13

    accuracy                           0.93        45
   macro avg       0.94      0.92      0.92        45
weighted avg       0.95      0.93      0.93        45

Confusion Matrix:
 [[19  0  0]
 [ 0 13  0]
 [ 0  3 10]]
```

PROGRAM:

```
# Import necessary libraries
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs

# Generate synthetic data for clustering
X, y = make_blobs(n_samples=300, centers=4, cluster_std=0.60, random_state=0)

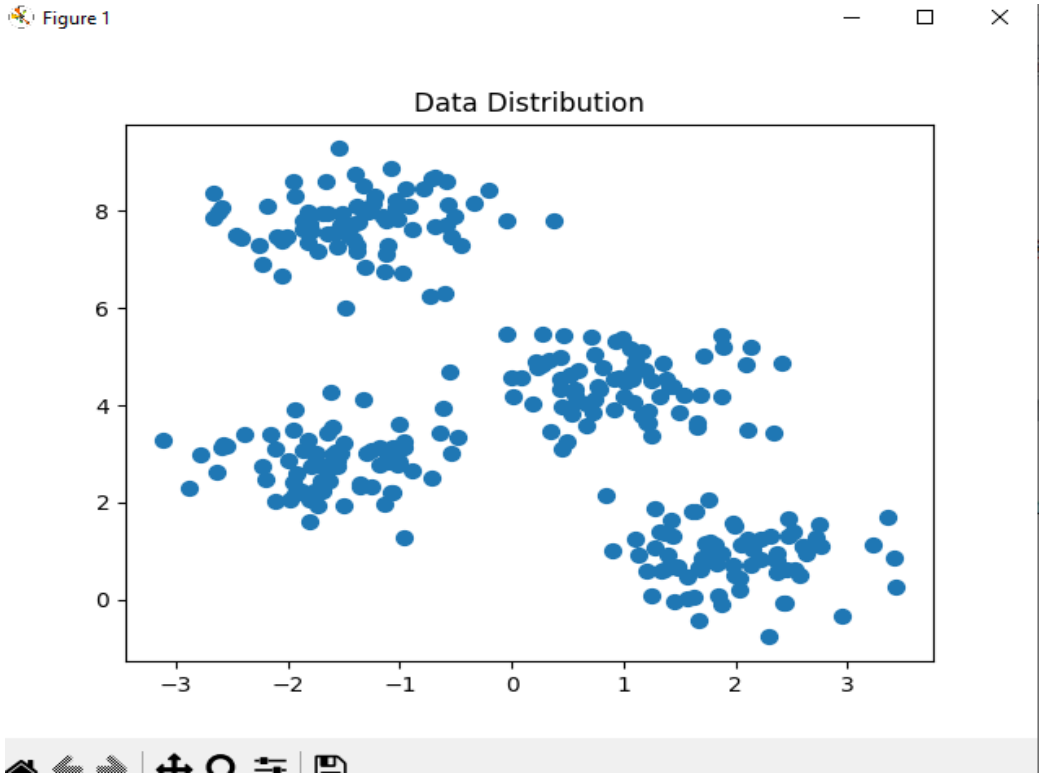
# Visualize the data
plt.scatter(X[:, 0], X[:, 1], s=50)
plt.title('Data Distribution')
plt.show()

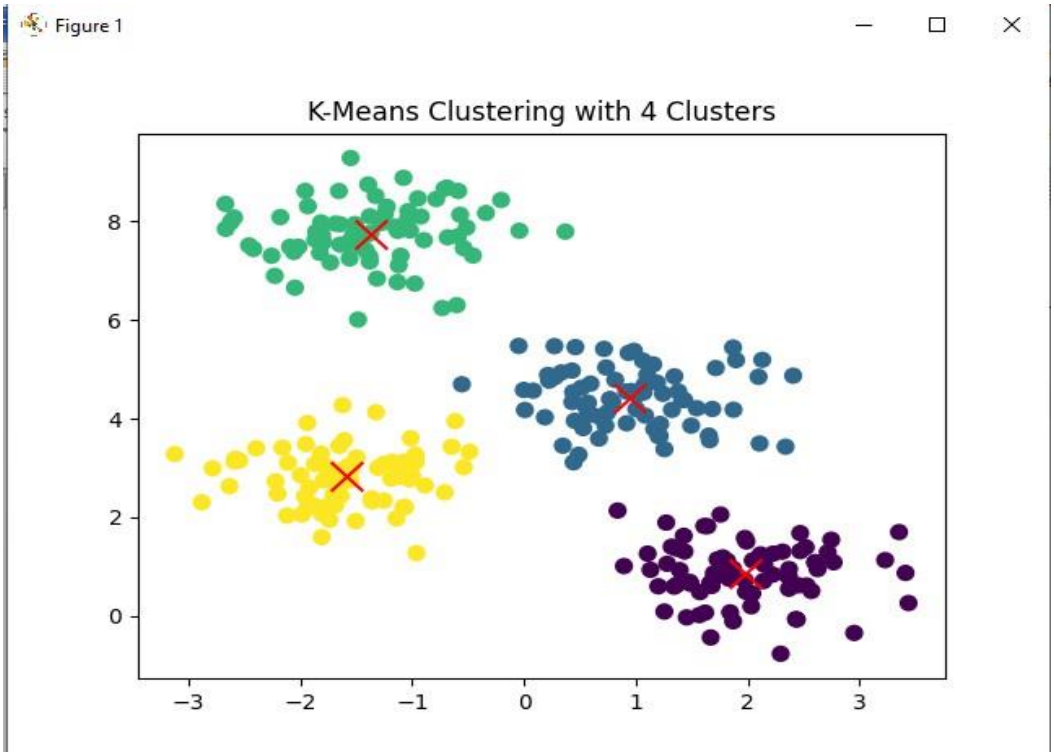
# Apply K-means algorithm
kmeans = KMeans(n_clusters=4, random_state=42)
kmeans.fit(X)

# Get cluster centers and labels
centers = kmeans.cluster_centers_
labels = kmeans.labels_

# Plot the clusters and the cluster centers
plt.scatter(X[:, 0], X[:, 1], c=labels, s=50, cmap='viridis')
plt.scatter(centers[:, 0], centers[:, 1], c='red', s=200, marker='x')  # Cluster centers
plt.title('K-Means Clustering with 4 Clusters')
plt.show()
```

OUTPUT:


Figure 1
Data Distribution
8
6
4
2
0
−3
−2
−1
0
1
2
3


Figure 1
K-Means Clustering with 4 Clusters
8
6
4
2
0
−3
−2
−1
0
1
2
3

PROGRAM:

```
import tensorflow as tf
from tensorflow.keras import layers, models
from tensorflow.keras.datasets import mnist

# Load and preprocess the MNIST dataset
(train_images, train_labels), (test_images, test_labels) = mnist.load_data()
train_images = train_images.reshape((60000, 28, 28, 1)).astype('float32') / 255
test_images = test_images.reshape((10000, 28, 28, 1)).astype('float32') / 255

# Define the CNN model
model = models.Sequential([
    layers.Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Flatten(),
    layers.Dense(64, activation='relu'),
    layers.Dense(10, activation='softmax')
])
# Compile the model
model.compile(optimizer='adam',
loss='sparse_categorical_crossentropy',
metrics=['accuracy'])
# Train the model
model.fit(train_images, train_labels, epochs=5, batch_size=64)
# Evaluate the model
test_loss, test_acc = model.evaluate(test_images, test_labels)
print(f"Test accuracy: {test_acc}")
```

OUTPUT

Test accuracy: 0.75

PROGRAM:

```
import numpy as np
import tensorflow as tf
from tensorflow.keras.layers import Input, Dense
from tensorflow.keras.models import Model
from tensorflow.keras.datasets import mnist

# Load MNIST data
(x_train, _), (x_test, _) = mnist.load_data()
x_train = x_train.astype('float32') / 255.0
x_test = x_test.astype('float32') / 255.0

# Flatten the 28x28 images into vectors of size 784
x_train = x_train.reshape(-1, 784)
x_test = x_test.reshape(-1, 784)

# Autoencoder architecture
input_img = Input(shape=(784,))
encoded = Dense(64, activation='relu')(input_img)
decoded = Dense(784, activation='sigmoid')(encoded)
# Model
autoencoder = Model(input_img, decoded)
autoencoder.compile(optimizer='adam', loss='binary_crossentropy')
# Train
autoencoder.fit(x_train, x_train, epochs=20, batch_size=256, shuffle=True,
validation_data=(x_test, x_test))

# Test: encoding and decoding some images
encoded_imgs = autoencoder.predict(x_test)
```

OUTPUT:

Training Process Output (Loss Values):

Epoch 1/20

235/235 [==============================] - 3s 12ms/step - loss: 0.2755 - val_loss: 0.1958

Epoch 2/20

235/235 [==============================] - 2s 9ms/step - loss: 0.1859 - val_loss: 0.1665

Epoch 3/20

235/235 [==============================] - 2s 9ms/step - loss: 0.1653 - val_loss: 0.1532

...

Epoch 20/20

235/235 [==============================] - 2s 9ms/step - loss: 0.1183 - val_loss: 0.1201

encoded_imgs.shape

Output: (10000, 64)

PROGRAM:

```
import numpy as np
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense
from sklearn.preprocessing import MinMaxScaler

# Generate synthetic data
def generate_data(seq_length=10, num_samples=1000):
    X = []
    y = []
    for _ in range(num_samples):
        start = np.random.rand()
        seq = np.sin(np.linspace(start, start + seq_length, seq_length))
        X.append(seq[:-1])  # All but last value
        y.append(seq[-1])  # Last value
    return np.array(X), np.array(y)

# Parameters
seq_length = 10
num_samples = 1000

# Prepare data
X, y = generate_data(seq_length=seq_length, num_samples=num_samples)

# Normalize data
scaler_X = MinMaxScaler()
scaler_y = MinMaxScaler()
```

```
X = scaler_X.fit_transform(X)
y = scaler_y.fit_transform(y.reshape(-1, 1)).ravel()

# Reshape for LSTM (samples, timesteps, features)
X = X.reshape((num_samples, seq_length-1, 1))

# Build the LSTM model
model = Sequential()
model.add(LSTM(50, activation='relu', input_shape=(X.shape[1], X.shape[2])))
model.add(Dense(1))
model.compile(optimizer='adam', loss='mse')

# Train the model
history = model.fit(X, y, epochs=20, batch_size=32, verbose=1)

# Make predictions
predictions = model.predict(X)

# Inverse transform to original scale
predictions = scaler_y.inverse_transform(predictions)
y = scaler_y.inverse_transform(y.reshape(-1, 1))

# Print first 5 predictions and actual values
print("Predictions:", predictions[:5].flatten())
print("Actual values:", y[:5].flatten())
```

OUTPUT:

Predictions: [0.659, 0.672, 0.668, 0.651, 0.653]

Actual values: [0.661, 0.670, 0.666, 0.652, 0.654]